

Wine Recommendation Questionnaire - Rule Engine Handbook

This handbook documents the architecture, current implementation, supported rule types, future extensions and best practices for maintaining the Laravel + Blade + JavaScript questionnaire.

1. Overall Architecture

Frontend (Blade + QuestionnaireEngine) renders questions.
Laravel Controller loads questions and questionnaire rules.
QuestionnaireRuleService returns rules.
Rules live in config/questionnaire_rules.php.
QuestionnaireEngine processes every question before rendering.
Product Matching runs after questionnaire completion.
SurpriseMe is replaced before matching.

2. Data Flow

Database Questions -> Controller -> RuleService -> Blade (questions + rules)
-> QuestionnaireEngine.process()
-> filterOptions()
-> applyHideOptionRule()
-> shouldSkipQuestion()
-> User Response
-> Product Matching
-> Recommendation

3. Rule Structure

Every rule contains:
source : triggering question_key
target : affected question_key
action : behaviour
Additional keys depend on the action (mapping, when, options, value, etc.).

4. Action: filter

Purpose: Keep only matching options.

Example:
source = country
target = sub_region
action = filter

mapping:
France => [Loire Valley, Bordeaux, Burgundy]
Italy => [Tuscany, Veneto]

Frontend:
filterOptions()
looks up mapping[sourceAnswer] and keeps only those options.

5. Action: hide_option

Purpose: Hide specific options.

Example:

```
source = wine_type  
target = sub_region  
action = hide_option  
when = Red  
options = [Champagne]
```

Frontend:

applyHideOptionRule() removes listed options.

6. Action: skip_question

Purpose: Skip an entire question.

Example:

```
source = wine_type  
target = tannin  
action = skip_question  
when = White
```

Frontend:

shouldSkipQuestion() returns true.

7. Future Actions

show_question

Example:

```
source=wine_type  
target=sparkling_style  
action=show_question  
when=Sparkling
```

set_default

Automatically select a value.

clear_answer

Clear dependent answers when parent answer changes.

require

Make a question mandatory only under conditions.

disable_option

Display but prevent selection.

sort_options

Custom display order.

message

Show contextual help or warnings.

8. How to Add a New Rule

1. Decide if an existing action solves the requirement.
2. Add the rule in config/questionnaire_rules.php.
3. If using an existing action, no engine changes are required.
4. If introducing a new action:
 - Add a new case inside QuestionnaireEngine.
 - Create a helper function if needed.
 - Test thoroughly.

9. QuestionnaireEngine Overview

Current methods:

```
getSelectedValues()  
process()  
filterOptions()  
applyHideOptionRule()  
shouldSkipQuestion()
```

Recommended pattern:

```
process()  
-> action dispatcher  
-> helper methods  
-> return processed question
```

10. SurpriseMe

SurpriseMe is intentionally NOT handled by the Rule Engine.

The questionnaire stores 'SurpriseMe'.

replaceSurpriseMeAnswers() replaces it with a random valid answer before product matching.

11. Best Practices

Always use question_key.

Never use question text.

Database option text must exactly match questionnaire_rules.php.

Keep business rules in config, not JavaScript.

Keep JavaScript generic.

Test every rule independently before combining rules.

12. Debug Checklist

If a rule fails:

- Is question_key correct?
- Is source answer present?
- Does DB text exactly match rule text?
- Is the rule under the correct template?
- Is the action implemented?
- Is the question receiving options from Laravel?

13. Current Supported Actions Summary

filter - COMPLETE
hide_option - COMPLETE
skip_question - COMPLETE

Engine is stable and can be extended with new actions without changing existing rules.